

DevTown Bootcamp Project Report



Realtime WebSocket Chat Application

Submitted By: Badal Singh

Bootcamp: AI Recommendation – Teaching Machines to Pick Your Next Movie

Technologies: React.js • Tailwind CSS • Node.js • Express.js • Socket.io

1.Introduction

Real-time communication has become a core requirement in modern web applications, from chat systems and collaborative tools to live dashboards and multiplayer experiences. This project, **Realtime WebSocket-Based Chat Application**, is designed to demonstrate the practical use of **bidirectional, event-driven communication** using **Socket.io** on top of WebSockets. The aim of this system is to build a functional and visually appealing chat interface where multiple users can join a shared conversation space, exchange messages instantly, and view live updates such as online users and typing indicators without any page reload.

The project leverages a modern, industry-relevant technology stack:

React.js for responsive and component-driven UI

Tailwind CSS for fast, modern, utility-based styling

Node.js + Express.js for server-side environment

Socket.io for establishing low-latency WebSocket communication

This application serves as a practical demonstration of how full-duplex communication works, how events are handled in real time, and how frontend applications dynamically render UI based on live server updates. It aligns closely with the concepts taught in the **DevTown AI Recommendation Bootcamp**, reinforcing skills essential for real-world software engineering roles.

Project Title

Realtime Communication: WebSocket-Based Chat Application

Objective

The objective of this project is to design and implement a real-time web chat application using WebSockets. The system should allow multiple users to join a common chat room, exchange messages instantly without page refresh, and demonstrate how event-driven communication works between a browser client and a Node.js server.

Technology Stack Used

Frontend: React, JavaScript, HTML, CSS (utility-style classes)

Backend: Node.js, Express.js, Socket.io

Communication Protocol: WebSocket (via Socket.io)

Development Tools: npm, nodemon, Visual Studio Code, modern web browser

2. Terminal Execution Proof

Backend server started using nodemon – WebSocket port active.

```
PS E:\Chat App\realtime-chat-server> npm run dev
```

```
> realtime-chat-server@1.0.0 dev
```

```
> nodemon server.js
```

```
[nodemon] 2.0.22
```

```
[nodemon] to restart at any time, enter `rs`
```

```
[nodemon] watching path(s): *.*
```

```
[nodemon] watching extensions: js,mjs,json
```

```
[nodemon] starting `node server.js`
```

```
Server on 3000
```

```
New client: 296Vi0jfaQfQ_dYZAAAC
```

```
New client: Nmk9BMS6cNC39WGpAAAD
```

```
New client: 7Mpdd0spP1VDVEx9AAAF
```

```
New client: -YlgnCyBzWv-DDR2AAAH
```

```
New client: k3Agu7Jm0MnwmngTLAAAJ
```

```
New client: pAh7dAH5PAWKCy14AAAL
```

Frontend React server compiled successfully and running on port 3001.

```
PS E:\Chat App\realtime-chat-client-tailwind> npm start
```

```
> realtime-chat-client-tailwind@1.0.0 start
```

```
> react-scripts start
```

```
✓ Something is already running on port 3000.
```

```
Would you like to run the app on another port instead? ... yes
```

```
(node:30832) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option .
```

```
(Use `node --trace-deprecation ...` to show where the warning was created)
```

```
(node:30832) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option .
```

```
Starting the development server...
```

```
Compiled successfully!
```

```
You can now view realtime-chat-client-tailwind in the browser.
```

```
Local: http://localhost:3001
```

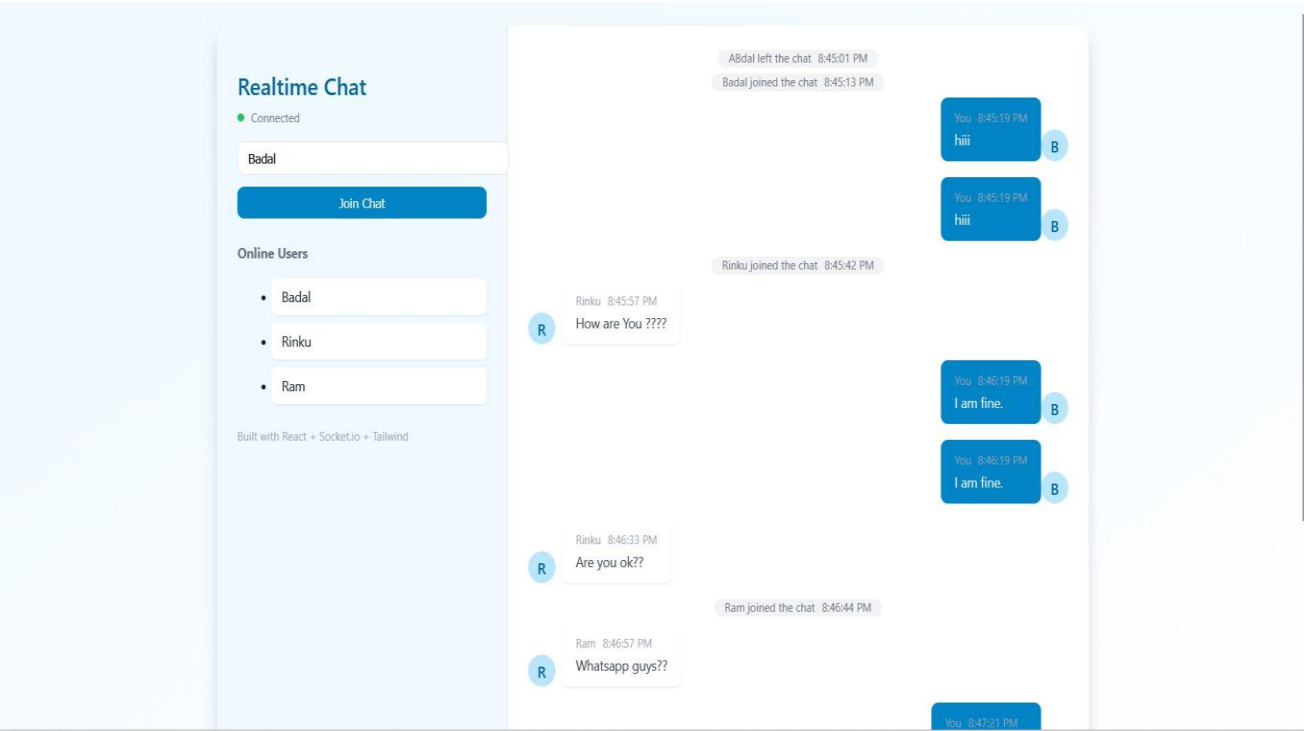
```
On Your Network: http://192.168.29.193:3001
```

```
Note that the development build is not optimized.
```

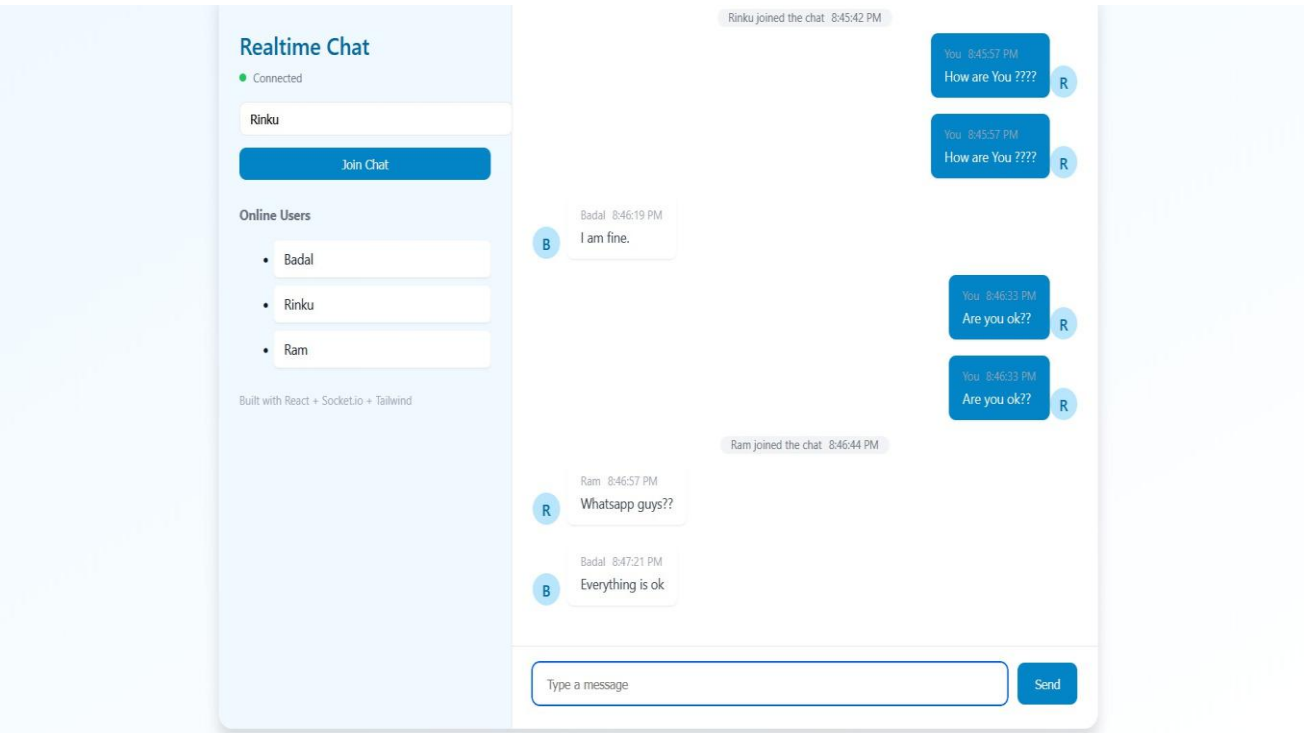
```
To create a production build, use npm run build.
```

```
webpack compiled successfully
```

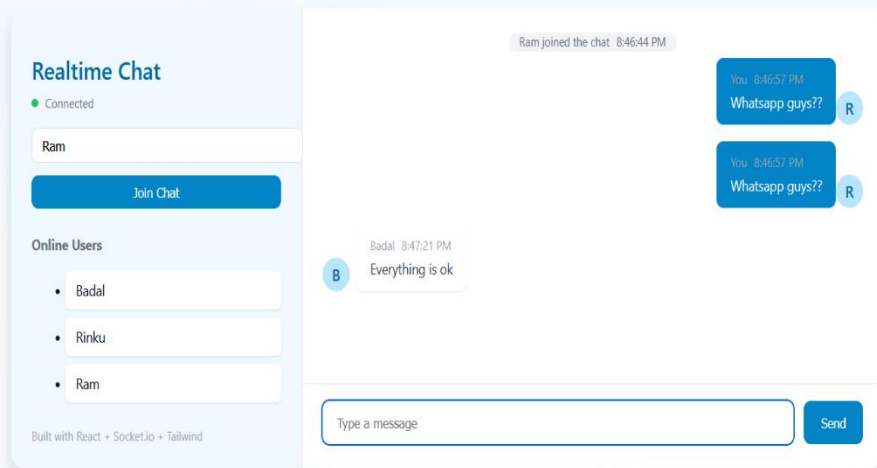
Browser UI screenshot showing chat interface.



Browser UI screenshot showing live messages.



Browser UI screenshot showing multiple users chatting.



3. Code Appendix

server.js

```
const express = require("express");
const http = require("http");
const cors = require("cors");
const { Server } = require("socket.io");

const app = express();
app.use(cors());
app.use(express.json());

const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*", methods: ["GET", "POST"] } });
let users = {};

io.on("connection", (socket) => {
  console.log("New client:", socket.id);

  socket.on("join", (name) => {
    const finalName = name || "Anonymous";

    users[socket.id] = finalName;
    io.emit("userList", Object.values(users));
    io.emit("systemMessage", {
      message: `${finalName} joined the chat`,
      time: new Date().toLocaleTimeString(),
    });
  });

  socket.on("sendMessage", (payload) => {
    io.emit("receiveMessage", payload);
  });

  socket.on("typing", (data) => {
    socket.broadcast.emit("userTyping", data);
  });
});
```

```
socket.on("disconnect", () => {  
  const name = users[socket.id];  
  delete users[socket.id];  
  io.emit("userList", Object.values(users));  
  if (name) {  
    io.emit("systemMessage", {  
      message: `${name} left the chat`,  
      time: new Date().toLocaleTimeString(),  
    });  
  }  
});  
});
```

```
server.listen(3000, ()=> console.log("Server on 3000"));
```

App.jsx

```
import React, { useEffect, useState, useRef } from 'react';
import { io } from 'socket.io-client';

const SOCKET_SERVER_URL = "http://localhost:3000";
const socket = io(SOCKET_SERVER_URL);

export default function App() {
  const [name, setName] = useState("");
  const [message, setMessage] = useState("");
  const [chat, setChat] = useState([]);
  const [users, setUsers] = useState([]);
  const [typingMsg, setTypingMsg] = useState("");
  const [isConnected, setIsConnected] = useState(socket.connected);
  const messagesEndRef = useRef(null);

  useEffect(() => {
    const handleConnect = () => setIsConnected(true);
    const handleDisconnect = () => setIsConnected(false);

    socket.on('connect', handleConnect);
    socket.on('disconnect', handleDisconnect);
    socket.on('receiveMessage', (payload) =>
setChat((p)=>[...p,payload]));
    socket.on('userList', (list) => setUsers(list));
    socket.on('userTyping', (d) => setTypingMsg(d.typing ? `${d.name}
is typing...` : ""));
```

```
socket.on('systemMessage', (payload) =>
  setChat((p)=>[...p, { ...payload, system: true }])
);
```

```
return () => {
  socket.off('connect', handleConnect);
  socket.off('disconnect', handleDisconnect);
  socket.off('receiveMessage');
  socket.off('userList');
  socket.off('userTyping');
  socket.off('systemMessage');
};
}, []);
```

```
useEffect(()=>{ messagesEndRef.current?.scrollIntoView({ behavior:
'smooth' }); }, [chat]);
```

```
const joinChat = () => {
  if (!name.trim()) return alert('Enter your name');
  socket.emit('join', name);
};
```

```
const sendMessage = () => {
  if (!message.trim()) return;
  const payload = { name, message, time: new
Date().toLocaleTimeString() };
  socket.emit('sendMessage', payload);
  setChat((p)=>[...p,payload]);
  setMessage('');
  socket.emit('typing', { name, typing: false });
};
```

```

const handleTyping = (v) => {
  setMessage(v);
  socket.emit('typing', { name, typing: v.length>0 });
};

return (
  <div className="min-h-screen bg-gradient-to-br from-sky-50 to-white
flex items-center justify-center p-6">
    <div className="max-w-5xl w-full bg-white shadow-lg rounded-2xl
grid grid-cols-3 overflow-hidden">
      {/* Left - Sidebar */}
      <div className="col-span-1 bg-sky-50 p-6 border-r">
        <h2 className="text-2xl font-semibold text-sky-700 mb-
2">Realtime Chat</h2>
        <div className="flex items-center text-xs text-gray-500 mb-4">
          <span className={`w-2 h-2 rounded-full mr-2 ${isConnected ?
'bg-green-500' : 'bg-gray-400'}}`></span>
          {isConnected ? 'Connected' : 'Disconnected'}
        </div>
        <div className="space-y-3">
          <input className="w-full px-3 py-2 border rounded-lg"
placeholder="Your name"
value={name} onChange={(e)=>setName(e.target.value)} />
          <button className="w-full bg-sky-600 text-white py-2 rounded-lg
hover:bg-sky-700"
onClick={joinChat}>Join Chat</button>
        </div>
        <div className="mt-6">
          <h4 className="text-sm text-gray-500 mb-2">Online Users</h4>
          <ul className="space-y-2">
            {users.map((u,i)=>(
              <li key={i} className="text-sm bg-white px-3 py-2 rounded-md
shadow-sm">{u}</li>
            ))}
          </ul>
        </div>
      </div>
    </div>
  </div>

```

```

<div className="mt-6 text-xs text-gray-400">
  Built with React + Socket.io + Tailwind
</div>
</div>

{/* Middle - Messages */}
<div className="col-span-2 flex flex-col">
  <div className="flex-1 p-6 overflow-auto" style={{minHeight:
'300px'}}>
    {chat.map((c,i)=>{
      if (c.system) {
        return (
          <div key={i} className="mb-2 flex justify-center">
            <div className="px-3 py-1 rounded-full bg-gray-100 text-xs
text-gray-500 shadow-sm">
              {c.message}
              {c.time && <span className="ml-2">{c.time}</span>}
            </div>
          </div>
        );
      }

      const isMe = c.name === name;
      return (
        <div key={i} className={`mb-4 flex ${isMe ? 'justify-end' :
'justify-start'}`>
          <div className={`flex items-end space-x-2 max-w-md ${isMe ?
'flex-row-reverse' : ''}`>
            <div className="w-8 h-8 rounded-full bg-sky-200 flex items-
center justify-center text-sky-700 font-semibold">
              {c.name?.charAt(0)?.toUpperCase() || 'U'}
            </div>
            <div className={`px-4 py-3 rounded-lg shadow-sm ${isMe ?
'bg-sky-600 text-white' : 'bg-white text-gray-700'}`>
              <div className="text-xs text-gray-400">
                {isMe ? 'You' : c.name}
                <span className="ml-2">{c.time}</span>
              </div>
            </div>
          </div>
        </div>
      );
    })}
  </div>
</div>

```

```

<div className="mt-1 text-sm">{c.message}</div>
    </div>
  </div>
</div>
);
}}
<div ref={messagesEndRef} />
</div>

```

```

    <div className="px-6 pb-6 pt-2 border-t">
      <div className="text-sm text-gray-500 mb-2">{typingMsg}</div>
      <div className="flex gap-3">
        <input className="flex-1 px-4 py-3 border rounded-lg"
          placeholder="Type a message"
          value={message}
          onChange={(e)=>handleTyping(e.target.value)}
          onKeyDown={(e)=>{ if (e.key==='Enter') sendMessage(); }} />
        <button className="bg-sky-600 text-white px-5 py-3 rounded-lg hover:bg-sky-700"
          onClick={sendMessage}>Send</button>
      </div>
    </div>
  </div>
</div>
</div>
);
}

```

README.md

Realtime WebSocket Chat Application

A small **realtime group chat** built with **React**, **Node.js**, **Express**, and **Socket.io**, with an enhanced WhatsApp-style UI.

This project demonstrates how WebSockets enable bidirectional communication between browser clients and a Node.js server, including typing indicators, online user list, and join/leave notifications.

Project Structure

```
``text
Chat App/
  README.md      # This file
  README.txt     # Short quick-start notes
  realtime-chat-server/ # Node.js + Express + Socket.io backend
  realtime-chat-client-tailwind/ # React frontend with enhanced UI
````
```

### ### Backend: `realtime-chat-server`

- Node.js + Express HTTP server
- Socket.io WebSocket server
- In-memory user tracking
- Events handled:
  - `connection` / `disconnect`
  - `join` – register user, update online list, broadcast join message
  - `sendMessage` – broadcast chat messages to all clients
  - `typing` – broadcast typing status to other users

### ### Frontend: `realtime-chat-client-tailwind`

- React functional components with hooks (`useState`, `useEffect`, `useRef`)
- `socket.io-client` for WebSocket connection
- Enhanced, WhatsApp-style chat UI using utility-style CSS classes
- Features:
  - Left sidebar with connection status and online users
  - Right panel with message bubbles and system notifications
  - Auto-scroll to latest message

---

## ## Features

### ### Basic

- **\*\*Realtime group chat\*\*** – all connected users see new messages instantly.
- **\*\*Live updates without page refresh\*\*** – WebSocket connection keeps UI in sync.
- **\*\*Message timestamps\*\*** – each message displays the time it was sent.
- **\*\*Auto-scroll\*\*** – chat automatically scrolls to the latest message.

### ### Intermediate

- **\*\*Typing indicator\*\*** – shows when another user is typing.
- **\*\*Online users list\*\*** – updates whenever users join or leave.
- **\*\*Join/leave notifications\*\*** – system messages like "Badal joined the chat".
- **\*\*Connection status badge\*\*** – green dot when WebSocket is connected.

### ### Future Enhancements (Optional)

- Multiple chat rooms / channels
- Private one-to-one messaging
- Persisting messages in a database (e.g., MongoDB)
- Read receipts and message status (sent / delivered / seen)
- Emoji picker and media attachments

## ## Getting Started

### ### Prerequisites

- Node.js (LTS version recommended)
- npm

Open a terminal in the **\*\*project root\*\*** (`Chat App/`).

### ### 1. Start the Backend (Server)

```
``bash
cd realtime-chat-server
npm install
npm run dev
``
```

- Server runs at: `http://localhost:3000`
- On start you should see: `Server on 3000`

### ### 2. Start the Frontend (Client)

In a new terminal window/tab:

```
``bash
cd realtime-chat-client-tailwind
npm install
npm start
``
```

- Default React port is `3000`. Because the server already uses port 3000, React will prompt to use another port (e.g. `3001`).
- Answer **\*\*Y\*\*** to run the client on the new port.
- Open the browser at: `http://localhost:3001`

---

## ## How It Works (High-Level Architecture)

### 1. **\*\*Client connection\*\***

- React app uses ``socket.io-client`` to connect to ``http://localhost:3000``.
- Once connected, the client listens for events like ``receiveMessage``, ``userList``, ``userTyping``, and ``systemMessage``.

### 2. **\*\*Joining the chat\*\***

- User enters a name and clicks **\*\*Join Chat\*\***.
- Client emits a ``join`` event with the user name.
- Server stores the name, updates the online users list, and broadcasts a system message: ``"<name> joined the chat"``.

### 3. **\*\*Sending messages\*\***

- Client emits ``sendMessage`` with ``{ name, message, time }``.
- Server broadcasts this payload to all clients via ``receiveMessage``.
- Each client appends the message to its local chat state and re-renders.

### 4. **\*\*Typing indicator\*\***

- While typing, client emits ``typing`` with ``{ name, typing: true/false }``.
- Server broadcasts ``userTyping`` so others see messages like ``"Rinku is typing..."``.

### 5. **\*\*Disconnecting\*\***

- On disconnect, server removes the user from the in-memory list.
- Broadcasts updated ``userList`` and a system message ``"<name> left the chat"``.

---

## ## UI Overview

- **\*\*Sidebar (left)\*\***
  - App title: **\*\*Realtime Chat\*\***
  - Connection status dot (green/gray) + label (Connected / Disconnected)
  - Input for user name and **\*\*Join Chat\*\*** button
  - "Online Users" list showing all connected users
- **\*\*Chat area (right)\*\***
  - Centered system messages for joins/leaves
  - WhatsApp-style message bubbles:
    - Your messages on the **\*\*right\*\*** in blue bubbles labeled "You"
    - Other users' messages on the **\*\*left\*\*** in white bubbles
  - Timestamps shown in each bubble
  - Typing indicator above the input field
  - Message input bar with **\*\*Send\*\*** button

---

## ## Screenshots (Suggested for Report)

When preparing your project report or submission, include screenshots such as:

- Chat window with multiple users and messages
- Typing indicator visible
- Online users list
- Join/leave system messages at the top of the chat

Add them to your document with captions like **\*\*"Figure 1: Realtime chat window with three users"\*\***.

---

# Certificate

devtown



## CERTIFICATE OF APPRECIATION

Presented to

**BADAL SINGH**

In Grateful recognition of their continuing  
support to help grow our community

  
**ASHISH MODI**  
Co-Founder, DevTown

Scan the QR code to verify



or visit: <https://cert.devtown.in/verify/1otbVa>

**12 Dec 2025**  
Issued Date

The certificate affirms that Badal Singh has satisfactorily fulfilled the requirements outlined. This validation ensures its authenticity, having been duly verified and granted by Devtown.

## 4. Conclusion

The Realtime WebSocket Chat Application developed as part of the DevTown Bootcamp successfully demonstrates the power of modern web technologies in enabling seamless, low-latency communication between distributed users. Through the integration of **React.js**, **Node.js**, **Express.js**, **Socket.io**, and **Tailwind CSS**, the project showcases how real-time systems can be built efficiently using event-driven architectures.

This project allowed the exploration of essential concepts such as **bidirectional WebSocket communication**, **state management in React**, **server-side event handling**, and **dynamic UI rendering**. Implementing features like live messaging, typing indicators, system notifications, and online user tracking provided a deep understanding of how real-time collaborative applications are designed and optimized.

Beyond its functional capabilities, the project highlights important software engineering principles including modular code organization, user-centric interface design, performance considerations, and scalability planning. Even though the current implementation uses in-memory storage, it sets a strong foundation for future expansion into production-grade systems with database persistence, authentication, private rooms, and media support. Overall, this project not only reinforced core frontend and backend development skills but also demonstrated how technologies work together to create smooth, dynamic user experiences.

The insights gained from this implementation will be valuable for developing advanced real-time applications in the future—ranging from messaging platforms to collaborative tools, live dashboards, multiplayer systems, and more. This hands-on experience has strengthened both conceptual understanding and practical expertise, achieving the true goal of the DevTown Bootcamp.